

Comprehensive Reference: Multi-Task Circuit Discovery via Venn-Gate L0 Pruning and Uncertainty Weighting

1. The Core Pruning Mechanism

The foundation of this method is continuous, differentiable network pruning using **L0 Regularization**. Instead of post-training magnitude pruning, we learn a specific probability that a given component (attention head, MLP neuron, block, etc.) should remain active during the forward pass.

This is implemented via the **HardConcreteGate**, which utilizes the Hard Concrete distribution and a Straight-Through Estimator (STE).

Original Component Sparsity Loss

For any single set of gates, the sparsity penalty (which economically forces the network to close unused gates) is calculated as the expected L0 density:

$$\mathbb{E}[L_{\text{sparsity}}] = \lambda \times \frac{1}{N} \sum_{i=1}^N \sigma \left(\log \alpha_i - \beta \log \left(\frac{\gamma}{\gamma + \zeta} \right) \right)$$

- **$\log \alpha$** : The learnable parameter for each gate (updated via gradient descent).
 - **β, γ, ζ** : Constant stretch parameters (e.g., $\beta = 2/3, \gamma = -0.1, \zeta = 1.1$) that allow the sigmoid to reach absolute 0 or 1.
 - **λ** : The global sparsity weight defining the "budgetary cost" of keeping a node open.
-

2. The Problem with Post-Hoc Boolean Logic

If a mechanistic behavior requires taking the **Intersection (AND)** or **Union (OR)** of multiple sub-circuits discovered across different corruption templates (Task A and Task B), training two separate masks (Mask_A and Mask_B) and mathematically intersecting them post-hoc is fundamentally unreliable.

- The network may find redundant, alternate computational paths in separate training runs.
- The thresholding dynamics differ between independent runs.
- We cannot guarantee that the resulting overlap represents a true, structurally critical aggregator node rather than a statistical artifact.

The Solution: We must natively embed the Boolean operations into the differentiable topology of the network during a *single* training run, forcing the optimizer to isolate shared circuitry naturally.

3. The Venn-Gate Architecture

Instead of learning one mask per task, we reparameterize the gating mechanism to represent a Venn diagram. We initialize **three** distinct **HardConcreteGate** modules:

1. **g_{core}** : The shared intersection circuit (nodes critical to both tasks).

2. **$g_{\{A_only\}}$** : Nodes specific to Task A.
3. **$g_{\{B_only\}}$** : Nodes specific to Task B.

Before evaluating the network, we construct the "Effective Gates" for each forward pass using a Differentiable Logical OR (implemented via STE):

- **Effective Gate A:** $G_A = g_{\{core\}} \vee g_{\{A_only\}}$
- **Effective Gate B:** $G_B = g_{\{core\}} \vee g_{\{B_only\}}$

During a single training step, the batch is passed through the network twice. Forward Pass A uses G_A to mix the Clean stream with Corrupted Stream A. Forward Pass B uses G_B to mix the Clean stream with Corrupted Stream B.

4. The Loss Function & Economic Incentives

To force the optimizer to sort nodes into their correct Venn categories, we rely on the specific sums of the task losses and carefully tuned L0 economic incentives.

4a. The Base Task Losses (Faithfulness & Correctness)

For each task, the raw task loss ensures the pruned circuit mimics the unpruned baseline (Faithfulness via KL Divergence) and outputs the correct token over the distractor (Correctness via Margin Loss).

For Task A: $KL_A = \text{KL_Div}(\text{LogSoftmax}(\text{Logits}_A), \text{LogSoftmax}(\text{Golden_Logits}))$
 $\text{TaskLoss}_A = \text{ReLU}(4.0 - (\text{Logit}_A[\text{target}] - \text{Logit}_A[\text{distractor}]))$
 $\text{Loss}_A = (KL_A \times 1.5) + \text{TaskLoss}_A$

For Task B: *(Identical formulation against Corrupted Stream B)* $\text{Loss}_B = (KL_B \times 1.5) + \text{TaskLoss}_B$

4b. Hyperparameter Rules for Boolean Logic

By adjusting the λ (cost) parameters associated with the Venn buckets, we mathematically force gradient descent to naturally discover either the Intersection or the Union.

To find the INTERSECTION ("AND" Node / Shared Core):

- **Rule:** $\lambda_{\{specific\}} < \lambda_{\{core\}} < 2 \times \lambda_{\{specific\}}$
- *Example:* $\lambda_{\{specific\}} = 0.5$, $\lambda_{\{core\}} = 0.6$.
- *Mechanism:* If a node is required by both tasks, opening it in both specific buckets costs \$1.0\$. Opening it in the shared $g_{\{core\}}$ costs \$0.6\$. The optimizer greedily chooses the cheaper shared bucket. If a node is only needed by Task A, Task A refuses to pay \$0.6\$ for $g_{\{core\}}$ and safely places it in $g_{\{A_only\}}$ for \$0.5\$.

To find the TOTAL UNION ("OR" of the entire circuit):

- **Rule:** $\lambda_{\{core\}} < \lambda_{\{specific\}}$
- *Example:* $\lambda_{\{core\}} = 0.4$, $\lambda_{\{specific\}} = 0.5$.
- *Mechanism:* The shared bucket is unconditionally the cheapest option. All nodes required by *either* task will migrate into $g_{\{core\}}$, rendering the specific buckets completely empty.

5. The Veto Problem (Shortcut Learning)

The Vulnerability: Node-level pruning struggles to verify multi-hop logic if the network uses direct connections to the logits. If Corrupted Stream A allows the model to simply output a massive negative "Veto" vector directly to the final layer, the network will bypass intermediate "AND" aggregators entirely. If Task A doesn't strictly rely on the "AND" node, the node drops out of g_{core} .

The Solutions:

1. **Symmetric Counterfactuals:** Instead of corruptions that yield "Garbage/Failure", use corruptions that require the model to compute a *different, valid answer*. This forces the network to utilize aggregator nodes to synthesize conflicting bounds rather than just sounding a veto alarm.
2. **Edge Pruning:** Transitioning from Node Pruning to Edge Pruning (gating the Q, K, V projections between specific layers) removes the ability of early nodes to communicate directly with the logits, forcing the computational pathway through the expected aggregators.

6. The Magnitude Problem & Homoscedastic Uncertainty Weighting

The Vulnerability: If Loss_A naturally operates on a much larger scale (e.g., magnitude of \$10.0\$) than Loss_B (magnitude of \$0.1\$), the gradients from Task A will violently overpower the careful λ economics. Task A will forcefully purchase any node it wants in g_{core} , destroying the intersection logic.

The Solution: We must internally normalize the gradient pressure so both tasks exert equal voting power on the Venn gates, without relying on manual tuning or static scalars. To achieve this, we utilize **Homoscedastic Uncertainty Weighting**.

The Homoscedastic Uncertainty Formulation

Derived from: Kendall, A., Gal, Y., & Cipolla, R. (2018). "Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

This method introduces two brand new, learnable scalar parameters into the optimizer— σ_A and σ_B —which represent the internal "noise" or observational uncertainty of each respective task. The network uses backpropagation to dynamically scale the losses based on these parameters.

The Final Unified Loss Equation:
$$L_{\text{Total}} = \frac{1}{2\sigma_A^2} \text{Loss}_A + \frac{1}{2\sigma_B^2} \text{Loss}_B + \log(\sigma_A) + \log(\sigma_B) + L_{\text{Venn_Sparsity}}$$

Where $L_{\text{Venn_Sparsity}}$ is:
$$\lambda_{\text{core}} L_{\text{core}} + \lambda_{\text{specific}} L_{\text{A_only}} + \lambda_{\text{specific}} L_{\text{B_only}}$$

Why this works:

1. **Dynamic Scaling:** The network is heavily incentivized to increase σ_A to squash a massive Loss_A down to a manageable scale.
2. **Preventing Collapse:** The $\log(\sigma_A)$ and $\log(\sigma_B)$ terms act as a strict regularization penalty. This prevents the network from simply pushing the σ parameters to infinity to make the task losses zero.

3. **Gradient Equilibrium:** Gradient descent naturally finds an optimal equilibrium for σ_A and σ_B . By dividing the raw loss by $2\sigma^2$, both massive and tiny losses are perfectly normalized before their gradients flow backward into the **HardConcreteGate** parameters.
4. **Preserving L0 Economics:** Because the gradient magnitudes from $Loss_A$ and $Loss_B$ are now stabilized and balanced, the Venn diagram's L0 penalty budget (λ_{core}) vs $\lambda_{specific}$ regains absolute authority over where nodes are placed, mathematically guaranteeing a clean intersection.